NAZAR
Code Walkthrough

A guided, sequential tour of how NAZAR is built — a real slice of the code, then a plain-language explanation of what it does and how, one piece at a time. Written to be readable even if you are new to coding: jargon is explained the first time it appears.

Built by HD Gala at the Takshashila Institution.

How to read this document

If you have been 'vibe coding' — building by feel, prompting an AI, pasting snippets — you may not have met all the words below. Here is the whole vocabulary you need for this tour, in one place.

- **HTML** (a `.html` file) — the skeleton of a web page: the text, buttons and boxes. Think of it as the nouns.
- **CSS** (a `.css` file) — the styling: colours, fonts, spacing, layout. The adjectives. NAZAR keeps almost all of it in one shared `styles.css`.
- **JavaScript** (a `.js` file) — the behaviour: what happens when the page loads, when you click, when data arrives. The verbs. This document is mostly about the JS.
- **JSON** — 'JavaScript Object Notation', a plain-text way to write structured data as `{ "key": value }` pairs and `[ lists ]`. It is how data files and web responses are formatted.
- **The DOM** — the live, in-memory version of the page. JavaScript edits the DOM to change what you see (e.g. `document.getElementById('x')` grabs an element).
- **An API / an endpoint** — a web address you fetch data from, like CelesTrak's TLE feed.
- **TLE** — 'Two-Line Element set', NORAD's two-line text description of one orbit.
- **SGP4** — the standard math that turns a TLE + a time into a position. NAZAR runs it via a library called `satellite.js`.
- **async / await / Promise** — JavaScript's way of waiting for something slow (like a download) without freezing the page.
- **localStorage** — a small key-value store inside your browser that survives page reloads. NAZAR caches data there.
- **Canvas / WebGL** — a drawing surface in the page; WebGL is the fast 3-D kind used for the globes. **SVG** is the crisp 2-D vector kind used for the flat map's lines.
- **requestAnimationFrame** ('rAF') — asks the browser 'call me again on the next frame', the standard way to animate smoothly.

> **A recurring trick: ?v=NN**
> You will see scripts loaded as `2d-views.js?v=17`. The `?v=17` is a 'cache-buster'. Browsers cache files by their address; adding a version number changes the address so the browser fetches the fresh copy after an edit. Bump the number, and everyone gets the new code.

The shape of the project

NAZAR is a **static site**: a folder of files with no server and no build step. Each page is an `.html` file paired with a same-named `.js` file, and they all share `styles.css` and one data-loading helper. To run it locally you just serve the folder; to publish, you commit to GitHub and GitHub Pages hosts it automatically.

> **The most important shared file**
> `tle-loader.js` is loaded by every page that needs satellite data. It exposes one global object, `window.Argos`, with the functions everyone reuses. We start the walkthrough there because everything else builds on it.

Part 1 — The shared data layer (tle-loader.js)

The whole file is wrapped in a pattern that keeps its internal helpers private and exposes only a tidy set of functions:

```
window.Argos = (function () {
  const OBSERVER  = { lat: 28.6139, lon: 77.2090, alt: 0.216 }; // New Delhi (km)
  const EARTH_R_KM = 6371;
  // ... many private helpers ...
  return {
    OBSERVER, EARTH_R_KM,
    inferPurpose, parseTLE, propagate, makeSatrecs,
    fetchTLEs, fetchChinaSatcat,
  };
})();
```

The `(function(){ ... })()` shape is an **IIFE** — an 'Immediately-Invoked Function Expression'. It runs once, right away, and whatever it `returns` becomes `window.Argos`. Anything not returned (the helpers, the constants) stays hidden inside, so it can't clash with other code. This is a classic way to make a clean 'module' without any build tooling.

`parseTLE(text)` — turn raw text into records

The feed arrives as one long block of text, three lines per satellite: a name, then line 1, then line 2. This walks the lines and groups them:

```
function parseTLE(text) {
  const lines = text.replace(/\r/g, '').split('\n');
  const out = [];
  for (let i = 0; i &lt; lines.length - 2; i++) {
    if (lines[i+1] &amp;&amp; lines[i+1][0] === '1' &amp;&amp; lines[i+2] &amp;&amp; lines[i+2][0] ===
      const name = lines[i].trim();
      const noradId = parseInt(lines[i+1].slice(2, 7), 10);
      out.push({ name, l1: lines[i+1], l2: lines[i+2], noradId });
      i += 2;
    }
  }
  return out;
}
```

It recognises a record by the fact that line 1 starts with the character `'1'` and line 2 with `'2'`. The satellite's catalogue number (its NORAD id) always sits in columns 3–7 of line 1, so `.slice(2, 7)` pulls it out. The result is an array of small objects — exactly the JSON-like shape the rest of the app wants.

`makeSatrecs(tles)` — prepare each orbit for math

```
function makeSatrecs(tles) {
  const out = [], seen = new Set();
  for (const t of tles) {
    if (seen.has(t.noradId)) continue;        // the feed sometimes lists a sat twice
    seen.add(t.noradId);
    out.push({ name: t.name, noradId: t.noradId,
               rec: satellite.twoline2satrec(t.l1, t.l2) });
  }
  return out;
}
```

`satellite.twoline2satrec` is the library turning the two text lines into a **satrec** — a pre-chewed record the SGP4 math can run against quickly. A `Set` is used as a memory of NORAD ids already seen, so duplicates in the feed don't create double dots on the globe.

`propagate(rec, now)` — the heart of everything

```
function propagate(rec, now, observer = OBSERVER) {
  const pv = satellite.propagate(rec, now);      // position + velocity in space
  if (!pv || !pv.position) return null;
  const gmst = satellite.gstime(now);            // how far Earth has rotated
  const gd   = satellite.eciToGeodetic(pv.position, gmst);
  const ecf  = satellite.eciToEcf(pv.position, gmst);
  const look = satellite.ecfToLookAngles(obs, ecf);   // angles from the observer
  return {
    lat: satellite.degreesLat(gd.latitude),
    lon: satellite.degreesLong(gd.longitude),
    alt: gd.height,                                    // km above sea level
    az: ..., el: ..., range: ...                       // look-angles from the city
  };
}
```

Given a prepared orbit and a moment in time, this returns the satellite's latitude, longitude and altitude, plus the angles to it from an observer city. Two coordinate frames appear here and it is worth knowing them, because they come back later on the 2-D page:

- **ECI** ('Earth-Centred Inertial') — a frame fixed to the stars. The raw SGP4 output is in ECI; an orbit is a fixed ellipse here.
- **ECEF / geodetic** — a frame fixed to the *ground*, which spins with the Earth. Latitude/longitude live here. `gstime` (Greenwich Mean Sidereal Time) is the rotation angle that converts one to the other.

> **This is the payoff of the whole file**
> Nearly every visual in NAZAR is just this one function's answer, drawn differently: a dot on a globe, an arrow on a map, a ring in space. Learn `propagate` and you understand 80% of the site.

`fetchTLEs()` — the three-stage safety net

Downloading is slow and can fail, so this is an `async` function (it can 'await' slow steps). In order it tries: your browser's recent cache, then the live CelesTrak feed with a short 4-second timeout, then the snapshot bundled with the site:

```
async function fetchTLEs() {
  const cached = cacheGet('argos.tle.v2', CACHE_TTL.tle);
  if (cached) return { tles: cached, source: 'cache' };
  try {
    const ctrl = new AbortController();
    const timer = setTimeout(() =&gt; ctrl.abort(), 4000);   // don't hang forever
    const r = await fetch(TLE_URL, { signal: ctrl.signal });
    clearTimeout(timer);
    if (r.ok) { /* parse, cache, return source:'celestrak' */ }
  } catch (e) { /* fall through to the bundled file */ }
  const r2 = await fetch('data/active.tle', { cache: 'no-cache' });
  return { tles: parseTLE(await r2.text()), source: 'bundled' };
}
```

The `AbortController` + `setTimeout` pair is how you put a stop-watch on a download: if CelesTrak hasn't answered in 4 seconds, give up and use the bundled copy instead. The function always tells the caller which `source` won, so the page can label the data 'live', 'cached' or 'bundled'.

Part 2 — The 2-D ground-track page (2d-views.js)

This is the richest page, so it is the best teacher. It draws a flat map, a search box, an animated orbit, and two little 3-D globes — all fed by `propagate`.

The projection: longitude and latitude straight to x and y

```
const vx = lon =&gt; lon + 180;   // longitude -180..180  -&gt;  x 0..360
const vy = lat =&gt; 90  - lat;   // latitude   90..-90   -&gt;  y 0..180
```

The world map is 'equirectangular' — the simplest projection there is — so a position maps to the screen with plain arithmetic: shift longitude to start at 0, and flip latitude because screen-y grows downward. The whole track is drawn as an **SVG** overlay laid exactly on top of the NASA 'Blue Marble' image, so the dotted lines sit right on their coastlines.

Splitting the track at the date-line

A ground track that crosses longitude 180° would otherwise draw a straight streak all the way across the map. The fix is to start a new line segment whenever two samples jump more than 180° apart:

```
function segmentPath(pts) {
  let d = '', prevLon = null, started = false;
  for (const p of pts) {
    if (prevLon !== null &amp;&amp; Math.abs(p.lon - prevLon) &gt; 180) started = false;
    d += (started ? ' L ' : ' M ') + vx(p.lon) + ',' + vy(p.lat);
    started = true; prevLon = p.lon;
  }
  return d;
}
```

In an SVG path string, `M` means 'move the pen here without drawing' and `L` means 'draw a line to here'. So a big longitude jump emits a fresh `M`, lifting the pen instead of streaking across the map.

Reverse geocoding: what place is it over?

On hover, each 30-minute mark shows the country (or ocean) below the satellite. There is no server to ask, so the page carries the world's country outlines as data and does a **point-in-polygon** test — a classic 'ray casting' check of whether a point falls inside a shape — against every country, with a coarse ocean-name fallback. It is pure geometry running locally.

Two synced globes, fed from one place

Both little globes update through a single function, so they can never drift apart from the map or each other:

```
function setNowMarker(lat, lon, alt, t) {
  // ...move the 2-D dot + label...
  updateGlobe(lat, lon, alt, t);    // globe 1: keep the satellite centred
  updateGlobe2(lat, lon, alt, t);   // globe 2: spin the Earth at its true rate
}
```

Globe 2 — the 'True Earth Rotation' trick

The goal: the Earth spins at its real rate while the satellite traces a fixed orbit that goes in front of and behind the planet. The neat trick is to leave the satellite in Earth-fixed coordinates but quietly **orbit the camera at the Earth's rotation angle**. Seen from that moving camera, the Earth appears to spin and the orbit stands still in space:

```
const gmst = satellite.gstime(t);                // Earth's true rotation angle
globe2.pointOfView({ lat: 22, lng: -(gmst * RAD), altitude: 3.8 }, 0);
if (ring2Group) ring2Group.rotation.y = -gmst;   // spin the orbit ring to match
```

Because `gstime` is the genuine sidereal angle (a full turn per 23h56m), the spin is physically correct, and because it is driven by the animation's simulated time, the speed slider scales it automatically. To make eccentric orbits look eccentric, globe 2 uses a near-linear altitude scale instead of the compressed one the tiny centred globe uses:

```
function globe2AltFrac(alt) {          // perigee hugs the surface, apogee flung out
  const km = alt &gt; 0 ? alt : 400;
  return Math.min(1.0, 0.03 + km / 45000);
}
```

An orbit is built once as a loop of points, split into a solid tube for the arcs in front of the Earth and a dashed tube for the arcs behind it. Whether a point is 'behind' is a simple occlusion test: does the straight line from the camera to the point pass through the globe sphere?

Making the satellite blink when it hides

A continuous animation loop reads that same 'behind' flag and, while the dot is behind the Earth, blinks and swells it so you can't miss it:

```
function globe2BlinkTick(ts) {
  if (satMesh2 &amp;amp;&amp;amp; satMesh2.visible) {
    if (behind2) {
      const b = (Math.sin(ts / 1000 * Math.PI * 3) + 1) / 2;  // a 0..1 pulse
      satMesh2.scale.setScalar(1.4 + 0.25 * b);               // swell
      satMesh2.material.opacity = 0.3 + 0.7 * b;              // blink
    } else { satMesh2.scale.setScalar(1); satMesh2.material.opacity = 1; }
  }
  requestAnimationFrame(globe2BlinkTick);                     // ...and again next frame
}
```

`requestAnimationFrame` at the end is what makes it loop: it asks the browser to call the function again on the next screen refresh, forever, giving a smooth ~60-times-a-second animation without a timer.

Part 3 — Sixteen thousand satellites at once (viz3d.js)

Drawing every active satellite as its own 3-D object would bring a browser to its knees. The solution is an **InstancedMesh**: one sphere shape, drawn many thousands of times in a single instruction to the graphics card, each copy given its own position and colour.

```
const SAT_GEOM = new THREE.SphereGeometry(1.6, 8, 8);   // one small sphere
const SAT_MAT  = new THREE.MeshBasicMaterial({ color: 0xffffff, transparent: true });
const instMesh = new THREE.InstancedMesh(SAT_GEOM, SAT_MAT, 20000);  // room for 20k
globe.scene().add(instMesh);
```

Positions are refreshed every few seconds, but running SGP4 for 16,000 satellites in one go would cause a visible stutter. So the work is **chunked** across animation frames — a thousand satellites per frame — so the globe keeps turning smoothly while the maths catches up:

```
function propagateChunk() {
  const end = Math.min(chunkIdx + CHUNK_SIZE, allSats.length);
  for (let i = chunkIdx; i &lt; end; i++) {
    const r = propagate(allSats[i].rec, propNow);
    const p = globe.getCoords(r.lat, r.lon, r.alt / EARTH_R_KM);
    _mat.compose(_pos.set(p.x, p.y, p.z), _quat, _scale);
    instMesh.setMatrixAt(i, _mat);            // place instance i
    instMesh.setColorAt(i, ORBIT_COLOR[orbitClass(r.alt)]);
  }
  chunkIdx = end;
  if (chunkIdx &lt; allSats.length) requestAnimationFrame(propagateChunk); // next slice
}
```

Reusing the scratch objects `_mat/_pos/_quat/_scale` instead of creating new ones each time avoids generating tons of short-lived garbage for the browser to clean up — an easy-to-miss performance win when a loop runs 16,000 × 20 times a minute.

Knowing which dot the cursor is over

Because the spheres are tiny, instead of the usual 3-D ray-cast the page projects every satellite's position to screen pixels and picks the nearest one to the cursor — faster and more forgiving — while skipping any hidden behind the globe. Clicking (or choosing from the search box) then fades the whole swarm down and draws the selected orbit as a tube.

Part 4 — The rest, briefly

SAT-STATS (sat-stats.js) — remembering across visits

Its counts are cumulative, which means it must remember what it has seen. It keeps a running database in `localStorage` (that small in-browser store), merging in each fresh catalogue so numbers only grow. The graphs are drawn with the Chart.js library, and a modal shows the raw TLE text with a character-by-character decoder.

ChinRepo (chinrepo.js) — a filtered join

It takes the SATCAT rows whose owner is 'PRC', matches them to the live TLEs by catalogue number, and renders a filterable table. A 'join' like this — line up two lists by a shared key — is one of the most common data operations there is.

The reference pages

`compendium.html` and `china-sat-series.html` are self-contained: their text, styling and data all live in the one file. No live data, just carefully organised knowledge.

Part 5 — Shipping it, and keeping it fresh

Every local script tag carries a `?v=NN` version so browsers pick up edits (see the tip on page 2). Deploying is just a commit to the `main` branch; GitHub Pages rebuilds in under a minute.

The data stays current by itself. A scheduled **GitHub Action** runs every 6 hours, re-downloads the catalogue, and — only if something changed and it passes a sanity check — commits the fresh files back into the repository. That is why the bundled snapshot the site falls back to is never more than a few hours stale.

You now have the whole map

Download small text files (Part 1), turn each into a position with SGP4 (`propagate`), and draw that position as dots, arrows, rings and globes (Parts 2–3). A robot keeps the files fresh (Part 5). Everything else is detail — and now you know where to look for it.